Order Management System with Microservices

Training – Morogoro

---

1. What We Built

A simple order management system split into small services:

- **Products**: add, update and view products
- **Inventory**: add and deduct stock
- **Orders**: place and cancel orders

Each service runs on its own and has its own database.

---

2. Technologies We Used

- **Spring Boot**: to build each service
- **Spring Data JPA + MySQL**: to save data
- **REST API**: so other services and Postman can call each service
- **OpenFeign**: for services to talk to each other
- **Eureka**: so services can find each other by name
- **API Gateway**: one entry point for all requests
- **Resilience4j**: to protect the system when something fails

---

3. Our Services

Service	What it does
Eureka Server	Keeps a list of all running services
API Gateway	Receives all requests and sends them to the right service
AddUpdateProduct	Adds and updates products
FetchAllProduct	Lists products, and finds them by id or name
ProductInventory	Records stock ADD/DEDUCT with opening and closing quantity
OrderService	Places and cancels orders, and saves order items

---

4. How It Works

```
Postman ──► API Gateway ──► Service
                 │
```

```
            Eureka Server
        (finds the service by name)
```

- Every service registers itself with Eureka.
- The gateway asks Eureka where a service is, then forwards the request.
- Services call each other with Feign, using the service name instead of a fixed address.

---

5. Placing an Order

1. The customer sends an order with a product and quantity.
2. OrderService gets the product price from the product service.
3. OrderService checks the stock in the inventory service.
4. If there is enough stock, the inventory is deducted.
5. The order and its order items are saved.

Cancelling an order returns the quantity back to inventory.

---

6. Inventory Tracking

Every stock movement is recorded:

Product	Opening	Quantity	Closing	Action
Laptop	0	50	50	ADD
Laptop	50	2	48	DEDUCT

The closing quantity becomes the next opening quantity.

---

7. API Gateway and Logging

- All requests go through one address: `localhost:8080`
- Example: `localhost:8080/order-service/orders`
- The gateway logs every request: what came in, the status, and how long it took

---

8. Protecting the System (Resilience)

Pattern	Simple meaning
Circuit Breaker	Stop calling a service that keeps failing, then try again later
Retry	Try again if a call fails
Rate Limiter	Allow only 5 orders per second
Bulkhead	Allow only 3 orders processing at the same time
Time Limiter	Stop waiting if a call takes too long

When protection is triggered, the user gets a clear message instead of a crash.

---

9. Lessons Learnt

1. Small services are easier to build, test and change.
2. Eureka and the gateway remove fixed addresses, so services can move or scale easily.
3. Resilience patterns stop one failing service from bringing down the whole system.

---

10. How This Helps Our Institution

1. Our systems, such as stock, procurement and service requests, can be built as separate services.
2. One gateway gives a single, logged entry point, which makes monitoring easier.
3. Our services stay available even during high traffic or partial failures.

Thank You

Questions?